

Streaming Live RAG

Real-Time Incremental Retrieval, Multi-Intent Decomposition, and State-Preserving Answer Refinement

1. Executive Summary & Problem Overview

Standard Retrieval-Augmented Generation (RAG) systems operate in a static, batch-oriented turn cycle: a user finishes speaking or typing, submits a query, and waits while the system searches a knowledge base, synthesizes an answer, and responds.

In conversational voice systems and real-time support scenarios, this paradigm introduces unacceptable conversational latency and fails to reflect natural human communication:

- **High Conversational Latency:** Waiting for the user to complete a multi-sentence utterance before initiating search forces pauses of several seconds.
- **Compound / Multi-Intent Requests:** Real user utterances often package multiple implied needs into a single stream of speech (e.g., asking about room capacity, cancellation terms, and catering options together).
- **Late-Arriving Constraints:** Users naturally introduce clarifications or additional constraints mid-conversation (e.g., *"Actually, the trip was international"*). Traditional systems discard prior context or restart the pipeline from scratch rather than performing incremental refinement.

The Target Challenge

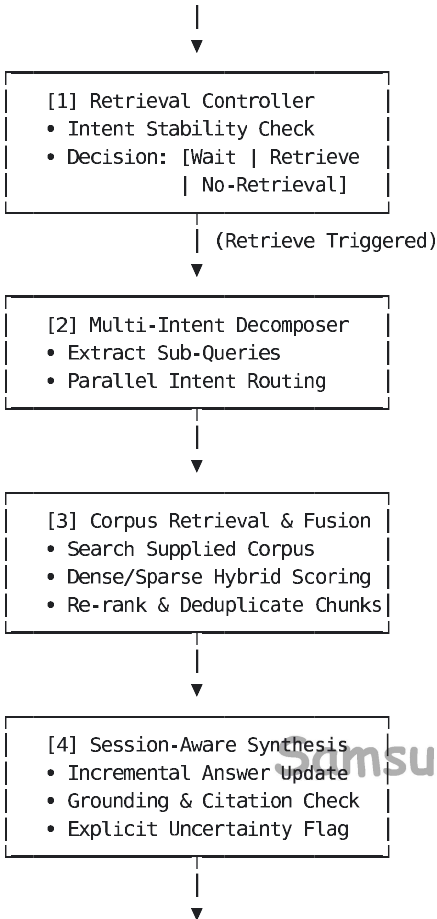
The objective is to design and build an event-driven **Streaming Live RAG Engine** that:

1. **Listens Incrementally:** Processes timestamped transcript chunks in real time, predicting retrieval intent before the user finishes speaking.
2. **Decomposes Multi-Intent Queries:** Identifies and parallelizes retrieval for multiple discrete sub-questions embedded within a single utterance.
3. **Refines Rather Than Restarts:** Selectively updates answers and citation graphs when late constraints arrive, preserving session state.
4. **Guarantees Corpus Grounding:** Enforces strict provenance and citation checks, returning explicit uncertainty indicators when evidence is insufficient.

Samsung Electronics

2. System Architecture & Pipeline

Incoming Stream: [Chunk 0.0s] → [Chunk 0.8s] → [Chunk 1.6s] → [Utterance End 2.1s]



Output: Streamed Answer + Grounded Citations + Observability Telemetry

Core Pipeline Components

Component	Functional Responsibilities	Core Engineering Challenge
1. Retrieval Controller	Evaluates incoming transcript fragments to decide when to retrieve, when to wait for semantic stabilization, and when to suppress search entirely (e.g., conversational formatting).	Balancing early retrieval latency gains against premature, noisy searches triggered by incomplete thoughts.
2. Multi-Intent Decomposition	Parses compound, unsegmented utterances into discrete, search-ready sub-queries.	Extracting distinct orthogonal questions without losing conversational context or over-fragmenting search.
3. Evidence Fusion & Reranking	Collects candidate chunks across all sub-queries, reconciles redundant evidence, and ranks snippets for maximum relevance and factual density.	Merging multi-source evidence without diluting context windows or introducing contradictory facts.
4. Session-Only Refinement	Applies late-arriving constraints (e.g., date adjustments, exceptions) directly onto existing answer states without clearing session context.	Tracking answer versions and mutating only affected claims rather than re-running full-corpus retrieval.
5. Observability Telemetry	Emits real-time event logs capturing timestamps, retrieval decisions, source mappings, version transitions, and token costs.	Maintaining clean, structured telemetry under sub-second streaming constraints.

3. Hard Engineering Rules & Constraints

To ensure rigorous benchmarking and evaluate algorithmic quality, the system must adhere to strict constraints:

- **Corpus Isolation:** All retrieved evidence must derive exclusively from the provided corpus. No external web scraping, third-party knowledge bases, or unindexed parametric model memory may be used to answer factual claims.
- **No Hardcoding / No Precomputation:** The benchmark replay evaluation is held-out and private. Prompts, queries, and canned responses must not be embedded in application code.
- **Rigorous Factual Grounding:** Every factual assertion must be attributed to verifiable corpus chunk IDs or section markers ([Doc_ID \$Section]). If the corpus lacks sufficient evidence to answer any sub-intent, the system must explicitly emit an uncertainty indicator or request targeted clarification.
- **Session-Bound State:** Cross-session profiling and persistent user tracking across independent test runs are prohibited. Memory is strictly ephemeral and scoped to the active conversation session.
- **Architectural Parsimony:** Multi-agent frameworks or complex orchestration pipelines are evaluated on cost-to-performance efficiency. Every added component must justify its latency and compute overhead.

4. Input & Output Event Specifications

The following scenarios demonstrate expected system behaviors across incremental streaming, late-detail refinement, and query suppression.

Example 1: Incremental Multi-Intent Utterance

A user speaks a complex, multi-part request. The system begins retrieval speculatively before the user finishes speaking, decomposes intents, and aggregates findings into a single grounded answer.

Stream Processing Timeline

Timestamp	Incoming Transcript Chunk	Controller Decision & System Action
0.0 s	"I need to plan a customer workshop in..."	Wait. Intent is incomplete and semantically unstable. No retrieval.
0.8 s	"...Pune for 30 people, and I need..."	Provisional Retrieve. Stable geographic and capacity entities identified. Issue early search for "Pune workshop venue capacity 30". Log retrieval_started.
1.6 s	"...the cancellation policy and the catering options."	Decompose & Parallel Retrieve. Deconstruct into three sub-queries: (1) venue capacity, (2) cancellation terms, and (3) catering options. Dispatch parallel searches and rerank candidate chunks.
2.1 s	[Utterance End]	Synthesize. Stream single unified response addressing all three sub-intents with citations, noting any unverified aspects.

Structured Output Event Record

```
{
  "retrieval_events": [
    { "timestamp_s": 0.8, "query": "Pune workshop venue capacity 30", "trigger": "provisional" },
    { "timestamp_s": 1.6, "query": "cancellation policy workshop venues Pune", "trigger": "multi_intent" },
    { "timestamp_s": 1.6, "query": "catering service options workshop Pune", "trigger": "multi_intent" }
  ],
  "sub_queries": [
    "venue capacity for 30 attendees in Pune",
    "cancellation terms and refund policies",
    "on-site and external catering options"
  ],
  "answer": "For a 30-person workshop in Pune, documented options include Venue A and Venue B. Venue A provide",
  "citations": [
    "Doc_12 $2",
    "Doc_31 $4",
    "Doc_09 $1"
  ],
  "uncertainty": "Catering accommodation policies for Venue A could not be verified from the retrieved corpus."
}
```

Example 2: Late-Arriving Detail (Refine, Do Not Restart)

A user introduces a constraint after receiving an initial answer. The system must update the answer version in-place, preserving established facts while querying only for the delta.

Interaction Step	User Utterance	Expected System Behavior & Response State
Step 1: Initial Request	"Summarize the travel reimbursement rule for an employee trip."	Retrieves base policy guidelines. Emits cited summary. Stores evidence context and Answer Version 1 in session memory.
Step 2: Late Detail	"The trip was international and the booking was made after travel."	Does not restart session. Recognizes modification to existing topic. Dispatches targeted queries for <i>international travel</i> and <i>post-travel booking exceptions</i> .
Step 3: Refined Response	(Generated Output)	"The standard reimbursement rule still applies. However, the late-booking exception requires senior director approval, and international travel introduces a mandatory foreign currency receipt verification requirement." (Preserves prior citations, adds delta citations, increments to Answer Version 2).

Example 3: Query Suppression (No Retrieval Required)

The system detects requests that modify formatting, style, or summarization of previously generated context, suppressing unneeded corpus queries.

User Input	Controller Evaluation	Expected Assistant Behavior
"Please repeat your last answer in two bullets."	retrieval_required: false reason: presentation_restructure	Transforms existing session context into two concise bullet points. No vector search or corpus queries executed. Retains prior citations without fabricating new ones.

5. Technical Evaluation Gates

Systems are evaluated against six automated and quantitative acceptance gates:

Gate	Criterion	Target Threshold	Validation Method
G1	Reproducibility	Pass / Fail	Container launches via a single command on a clean machine; automated replay suite completes without manual intervention.

Gate	Criterion	Target Threshold	Validation Method
G2	Early Retrieval	>= 80% of eligible queries	Retrieval commences prior to final transcript completion on held-out streaming prompts, maintaining low false-trigger rates on no-retrieval cases.
G3	Multi-Intent Identification	>= 70% of compound queries	Accurately identifies and isolates at least two distinct sub-intents in compound test utterances.
G4	Factual Grounding	>= 85% citation support	All sampled factual assertions are supported by cited corpus chunks; zero fabricated or hallucinated document IDs.
G5	Session Refinement	Verified state continuity	Late-arriving constraints narrow or update existing responses without clearing session state or re-executing full-corpus search.
G6	Telemetry & Observability	100% trace coverage	Structured logs or metrics dashboards capture execution timestamps, retrieval triggers, citations, answer version lineage, and token cost.

6. Common Technical Pitfalls

- 1. **Eager / Premature Retrieval on Noise:** Triggering vector search on every incremental token causes system thrashing, high compute costs, and noisy context windows. The controller must wait for semantic intent boundaries.
- 2. **Context Loss on Late Constraints:** Resetting the conversation state when a user introduces a clarification discards valuable retrieved context, doubling latency and causing disjointed replies.
- 3. **Citation Hallucination:** Generating fabricated citations ([Doc_999]) or citing chunks that do not explicitly contain the stated facts fails automated grounding gates.
- 4. **Ignoring Presentation-Only Turns:** Querying the vector database when the user simply asks to reformat, shorten, or translate prior output wastes tokens and risks introducing drift.
- 5. **Over-Fragmenting Sub-Queries:** Splitting a single simple question into multiple near-identical queries pollutes the reranker and exhausts token limits.

Samsung Electronics

7. Implementation & Architecture Roadmap

Phase 1: Framing & Foundation

- └─ Corpus audit, indexing, and chunking optimization
- └─ Setup of baseline retrieval pipeline (dense/sparse hybrid)
- └─ Definition of structured event schemas (Controller, Telemetry, Output)

Phase 2: Controller & Live Stream Simulation

- └─ Incremental transcript chunking simulator
- └─ Intent stability classifier & retrieval decision policy (Retrieve / Wait / Suppress)
- └─ Early retrieval timestamp logging and latency measurement

Phase 3: Multi-Intent Parsing & Evidence Fusion

- └─ Query decomposition module for compound requests
- └─ Parallelized corpus retrieval across sub-queries
- └─ Reciprocal Rank Fusion (RRF) and deduplication reranker

Phase 4: Session Refinement & State Management

- └─ Ephemeral session memory store
- └─ Answer delta engine: mutates only affected claims when late constraints arrive
- └─ Strict grounding verification and explicit uncertainty flagging

Phase 5: Telemetry, Benchmarking & Packaging

- └─ End-to-end telemetry instrumentation (timestamps, latencies, tokens, versions)
- └─ Benchmark execution across held-out streaming test sets
- └─ Single-command container packaging and deployment documentation

8. Engineering Deliverables Checklist

- ☐ **Reproducible Repository:** Source code, pinned dependency lockfiles, environment configuration templates, and one-command run instructions (docker compose up or clean CLI runner).
- ☐ **System Architecture Brief (<= 6 pages):** System design rationale, retrieval trigger logic, query decomposition strategy, data provenance, trade-offs, and failure mode mitigations.
- ☐ **Benchmarking & Evaluation Report:** Quantitative performance comparison against the baseline pipeline, documenting at least three analyzed edge-case failures and two architectural ablation experiments (e.g., hybrid vs. dense-only retrieval, rule-based vs. model-based controller).
- ☐ **System Demonstration Video (<= 5 minutes):** Walkthrough demonstrating early retrieval triggering, multi-intent decomposition, late-detail refinement, presentation query suppression, citation traceability, and runtime telemetry.
- ☐ **Telemetry & Observability Schema:** Structured logs capturing end-to-end request latencies, retrieval trigger events, answer version updates, and inference cost estimations.

Samsung Electronics